应用报告
SPRACL3–August 2019

Designing With the C2000™ Configurable Logic Block (CLB)

Nima Eskandari

摘要

C2000 可配置逻辑块（CLB）由一组可配置逻辑块组成，这些逻辑块通过软件互连，以实现自定义数字逻辑功能。本应用报告设计并测试了一个简单的自定义数字逻辑系统，并逐一检查和配置 CLB 模块中的各个可配置逻辑块，以实现所需的系统功能。

目录

1 引言 3
2 补充在线资料 3
3 设计概述 4
4 输入采样 5
5 在 FSM 子模块中实现状态机 6
6 生成 PWM 信号 9
7 修改 PWM 周期和占空比 12
8 完成的设计 15
9 输入 X-BAR、输出 X-BAR 和 CLB X-BAR 17
10 运行示例项目 20
11 总结 24
12 参考资料 24

插图目录

1 状态机框图 4
2 COUNTER0 配置 5
3 LUT0 和 LUT1 配置 6
4 FSM1 S0 卡诺图 7
5 FSM1 S1 卡诺图 7
6 FSM1 输出卡诺图 8
7 OUTLUT4 和 OUTLUT5 配置 8
8 FSM2 配置 10
9 COUNTER1 配置 10
10 FSM0 配置 11
11 OUTLUT0 和 OUTLUT2 配置 12
12 LUT2 配置 13
13 HLC 配置 14
14 完整设计框图 16
15 此实例中连接无效 17
16 CLB BOUNDARY 输入多路复用 18
17 TSM320F28379D LaunchPad 连接 20
18 在 Expressions 窗口中修改 PWM 周期和占空比 21
19 PWM 占空比 1000，周期 2000 22

Copyright © 2019, Texas Instruments Incorporated

20 示波器 — PWM 占空比 1000，周期 2000 23
21 PWM 占空比 3000，周期 4000 23
22 示波器波形——PWM 占空比 3000，周期 4000 24

表格目录

1 FSM 真值表 6

商标

C2000、LaunchPad 和 Code Composer Studio 是 Texas Instruments 的商标。

Copyright © 2019, Texas Instruments Incorporated

1 引言

为帮助读者理解 CLB，本文逐一介绍各个可配置逻辑块，并说明如何将它们组合使用。CLB 子系统包含若干相同的逻辑单元（tile）。F2837xD 的 CLB 子系统包含四个相同的 tile；其他器件中的 tile 数量可能更多或更少。每个 tile 包含以下模块：

- 4 输入查找表（LUT4）子模块
- 计数器子模块
- 有限状态机（FSM）子模块
- 3 输入输出查找表（Output LUT）子模块
- 高级控制器（HLC）子模块

本文设计了一个简单的 4 状态状态机，并使用 CLB 逻辑单元（tile）中的各个子模块，展示其功能及用法示例。

2 在线补充资料

有关特定 C2000 器件上的 CLB 模块，请参阅该器件的数据手册及相应的技术参考手册（TRM）。

本应用报告基于 TMS320F2837xD 系列器件编写。本文所用的数据手册和 TRM 如下：

- *TMS320F2837xD Dual-core Delfino™ Microcontrollers Datasheet*
- *TMS320F2837xD Dual-core Delfino™ Microcontrollers Technical Reference Manual*

TI E2E™ 社区还提供其他支持。

Copyright © 2019, Texas Instruments Incorporated

3 设计概述

本文采用的设计　　是一个简单的 4 状态状态机。4 个　　状态如下：

- 打开
- 正在关闭
- 关闭
- 正在打开

该设计类似于一个简单的车库门。状态机有两个外部输入用于交互：

- BOUNDARY IN0：按钮输入，根据当前状态打开或关闭车库门
- BOUNDARY IN1：传感器输入，用于指示关门或开门操作是否完成

图 1 展示了状态机的各个状态，以及外部输入如何触发状态转换。

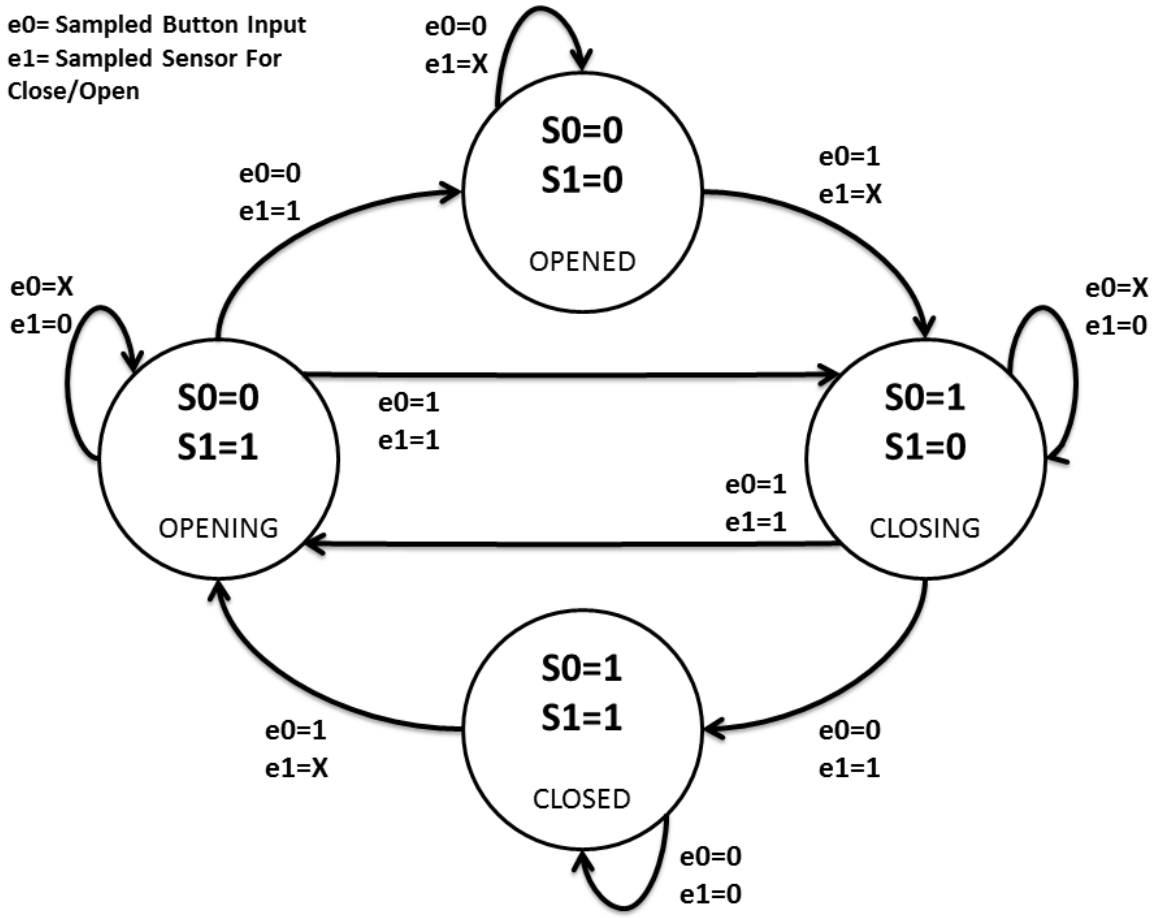


图 1. 状态机框图

当 BOUNDARY IN0（按钮输入）为 HIGH 时，系统会从“打开”或“关闭”状态转入“正在关闭”或“正在打开”状态。当 BOUNDARY IN1（传感器输入）为 HIGH 时，系统会从“正在打开”或“正在关闭”状态转入“打开”或“关闭”状态。如果 IN0 和 IN1 同时为 HIGH，系统会从“正在关闭”状态转入“正在打开”状态，或从“正在打开”状态转入“正在关闭”状态。其他输入组合不会使状态机发生状态转换。

Copyright © 2019, Texas Instruments Incorporated

4 输入采样

设计该系统时，首先对外部输入进行采样，以便更轻松地测试最终设计。BOUNDARY IN0 和 IN1 所用 GPIO 的原始输入不直接使用，而是通过计数器每 50,000 个时钟周期采样一次。可使用示波器观察轮询结果，例如显示从“正在关闭”到“正在打开”的状态转换；详见第 10 节。

计数器模块（COUNTER0）与 LUT0、LUT1 配合生成采样输入信号，随后由 FSM 子模块使用。采样逻辑需满足以下要求：

- COUNTER0 用于每 50000 个时钟周期生成一个信号（match1_val = 50000）
- 计数器始终使能（mode0 = 1），并向上计数（mode1 = 1）
- 在 MATCH1 事件发生时复位计数器
- 使用 LUT0 和 LUT1
- 将两个输入信号（BOUNDARY IN0 和 IN1）与 COUNTER0 的 MATCH1 事件信号 结合使用。这些信号用于生成采样输入，随后由 FSM 子模块使用。

图 2 和图 3 展示了这些要求，以及如何在示例项目的 SysConfig 设置中实现它们。

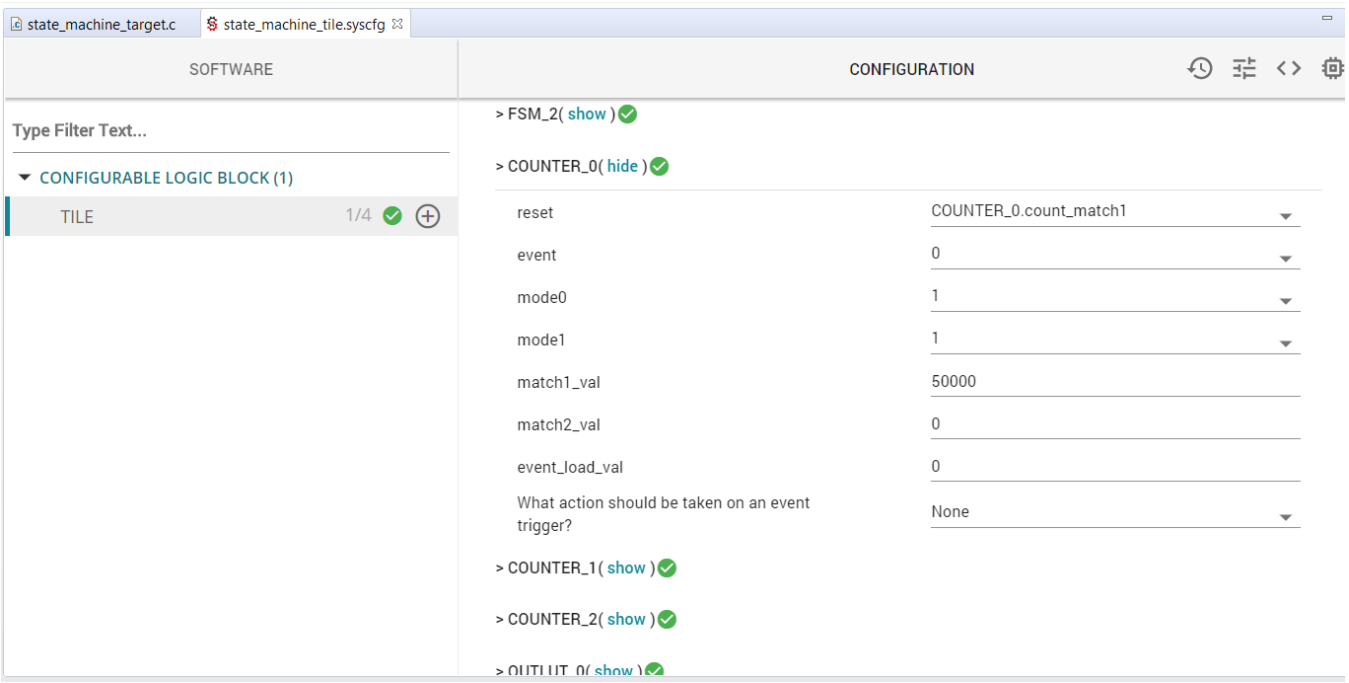


图 2. COUNTER0 配置

Copyright © 2019, Texas Instruments Incorporated

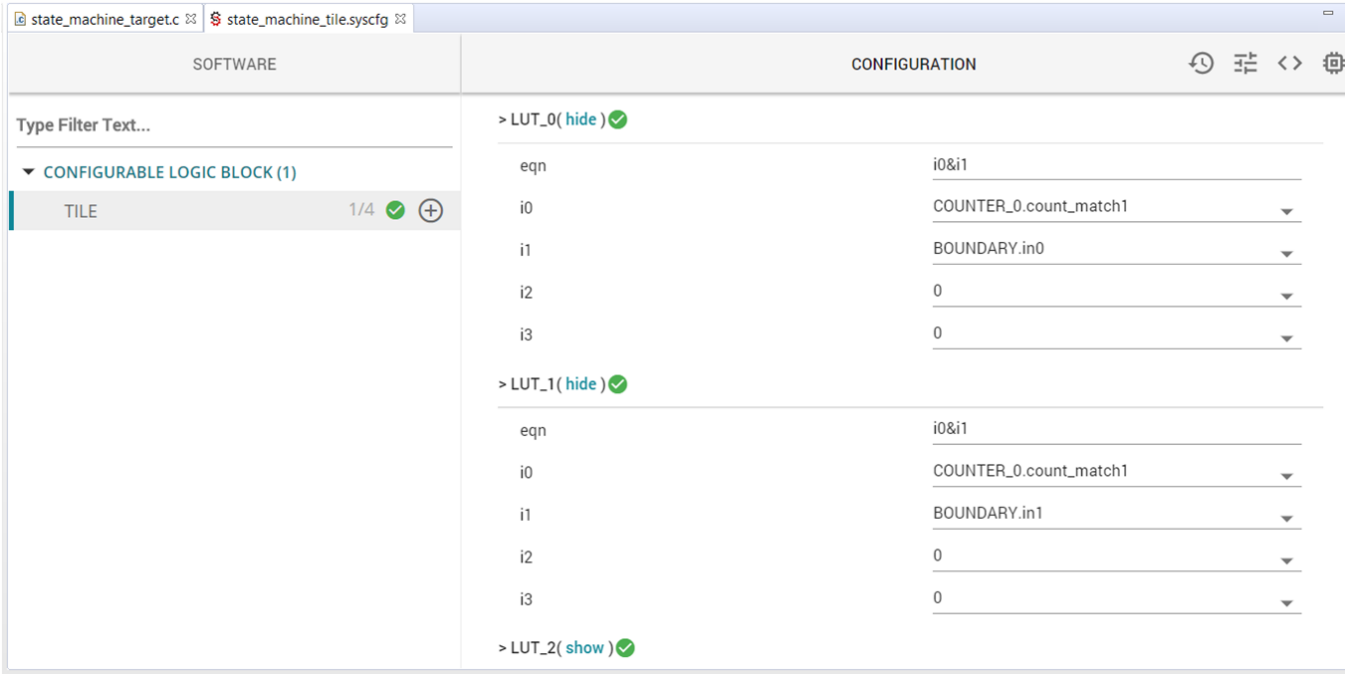


图 3. LUT0 和 LUT1 配置

5 在 FSM 子模块中实现状态机

图 1 所示状态机通过上一节生成的采样输入实现。首先，创建一张完整的真值表，列出输入、当前状态和状态机的下一状态。FSM 子模块还会生成一个输出信号。在本设计中，当系统状态切换为“正在打开”或“正在关闭”时，该输出信号置为 HIGH。后续设计会使用该输出信号对 PWM 信号进行门控。表 1 给出了完整的设计真值表。

表 1. FSM 真值表

s0	s1	e0	e1	s0 NEXT	s1 NEXT	OUTPUT
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	0	1	0	1	0	1
0	0	1	1	1	1	1
0	1	0	0	0	1	0
0	1	0	1	0	0	0
0	1	1	0	0	1	0
0	1	1	1	1	0	1
1	0	0	0	1	0	0
1	0	0	1	1	1	0
1	0	1	0	1	0	0
1	0	1	1	0	1	1
1	1	0	0	1	1	0
1	1	0	1	1	1	0
1	1	1	0	0	1	1
1	1	1	1	0	1	1

Copyright © 2019, Texas Instruments Incorporated

根据 FSM 真值表，分别为状态 s0、状态 s1 以及 FSM 输出绘制卡诺图，以求得相应的逻辑方程。

S0S1 \ e0e1	00	01	11	10
00	0	0	1	1
01	0	0	1	0
11	1	1	0	0
10	1	1	0	1

EQ S0 = (!e0&s0) | (e0&e1&!s0) | (!s0&!s1&e0) | (e0&!e1&!s1)

图 4. FSM1 S0 卡诺图

S0S1 \ e0e1	00	01	11	10
00	0	0	0	0
01	1	0	0	1
11	1	1	1	1
10	0	1	1	0

EQ S1 = (s1&!e1) | (s0&s1) | (s0&e1)

图 5. FSM1 S1 卡诺图

Copyright © 2019, Texas Instruments Incorporated

S0S1 \ e0e1	00	01	11	10
00	0	0	1	1
01	0	0	1	0
11	0	0	1	1
10	0	0	1	0

EQ OUTPUT = (e0&e1) | (!s0&!s1&e0) | (s0&s1&e0)

图 6. FSM1 输出卡诺图

用于测试本设计的 F28379D LaunchPad 会通过板载 LED 显示 FSM1 的 s0 和 s1。OUTLUT4 和 OUTLUT5 将这些信号从 FSM1 输出到 OUTPUT X-BAR，随后选用该 X-BAR 驱动 GPIO。GPIO34 和 GPIO31 的输出被拉低时，LaunchPad 上的 LED 会点亮。因此，在通过 OUTLUT4 和 OUTLUT5 将 s0 和 s1 从 CLB 输出之前，先对它们取反；这样当 s0 和 s1 为 HIGH 时，LED 即点亮。图 7 展示了 OUTLUT4 和 OUTLUT5 的 SysConfig 配置。

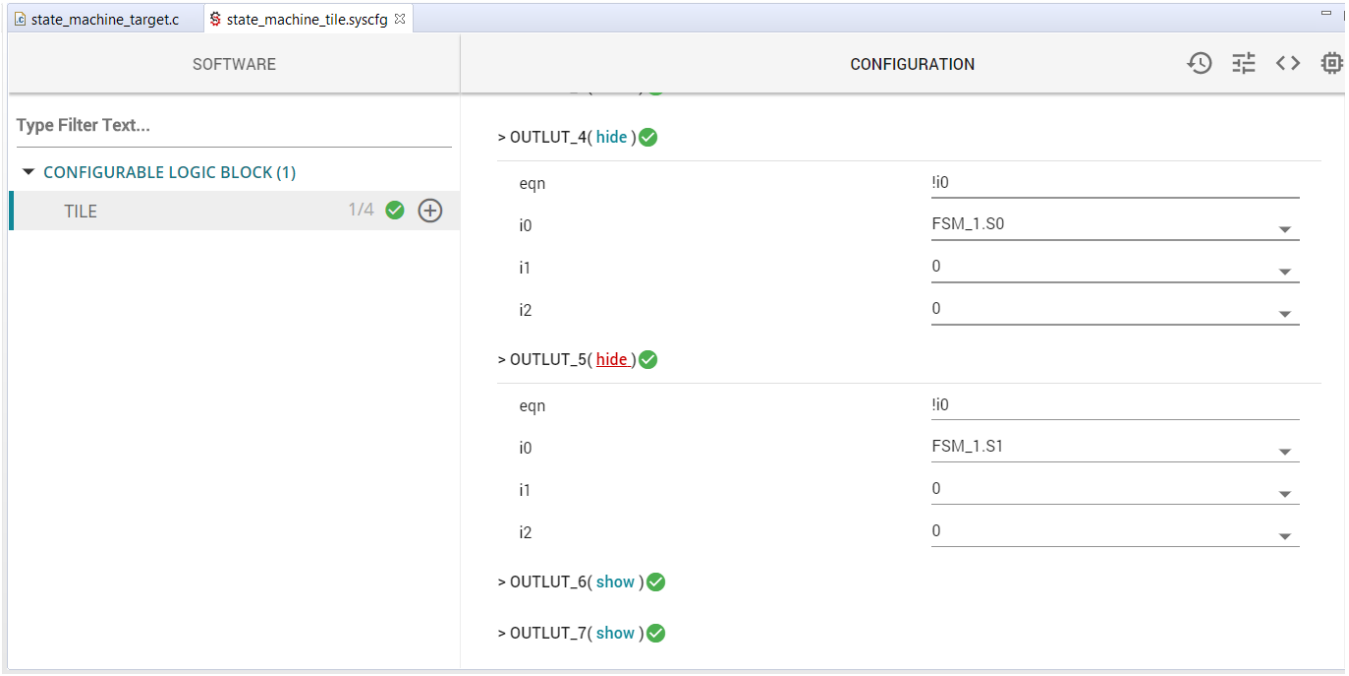


图 7. OUTLUT4 和 OUTLUT5 配置

Copyright © 2019, Texas Instruments Incorporated

6 生成 PWM 信号

CLB 可利用其可配置逻辑块生成 PWM 波形。假设需要在系统处于“正在打开”或“正在关闭”状态时生成 PWM 信号。本系统的 CLB 逻辑单元（tile）输出两路 PWM 信号：

- 正在打开状态：第一路 PWM 信号有效，其周期和占空比可调；第二路 PWM 信号保持为 LOW。
- 正在关闭状态：第一路 PWM 信号保持为 LOW，第二路 PWM 信号有效，且其周期和占空比可调。

FSM0、FSM2 和 COUNTER1 共同生成 PWM 信号。计数器子模块是 PWM 生成的核心部分，FSM0 和 FSM 2 则补全其余逻辑；这些功能共同实现 PWM 波形生成。

COUNTER1 配置为向下计数；计数器达到 ZERO 或指定匹配值时会生成相应信号。COUNTER1 的要求如下：

- 从 load_val 开始向下计数（mode1 = 0）。
- 发生 counter=ZERO 事件时，将 load_val 装载到计数器中。
- PWM 信号的周期由 COUNTER1 的 load_val 寄存器设定。
- 后续步骤将使用 counter=match1_val 事件生成的信号。
- PWM 信号的占空比由 COUNTER1 的 match1_val 寄存器设定。
- 计数器的使能信号为 mode0。仅当系统处于“正在打开”或“正在关闭”状态时，该信号才应有效（此计数器输入信号将在后续步骤中生成）。
- 检测到状态切换至“正在打开”或“正在关闭”时复位计时器（该信号已由 FSM1 OUTPUT 提供）。

PWM 的计时计数器现已配置完成。接下来要生成一个仅在系统处于“正在打开”或“正在关闭”状态时有效的信号。此处将 FSM2 配置为 3 输出 LUT，而不是有限状态机，以满足以下要求：

- S0：仅在“正在打开”状态有效
- S1：仅在“正在关闭”状态有效
- OUTPUT：“正在打开”或“正在关闭”状态下有效

FSM2 的输出作为 COUNTER1 的 mode0 输入。因此，只有系统处于“正在关闭”或“正在打开”状态时，COUNTER1 的 mode0 才会有效。

Copyright © 2019, Texas Instruments Incorporated

图 8 和图 9 分别展示了 FSM2 和 COUNTER1 的 SysConfig 配置。

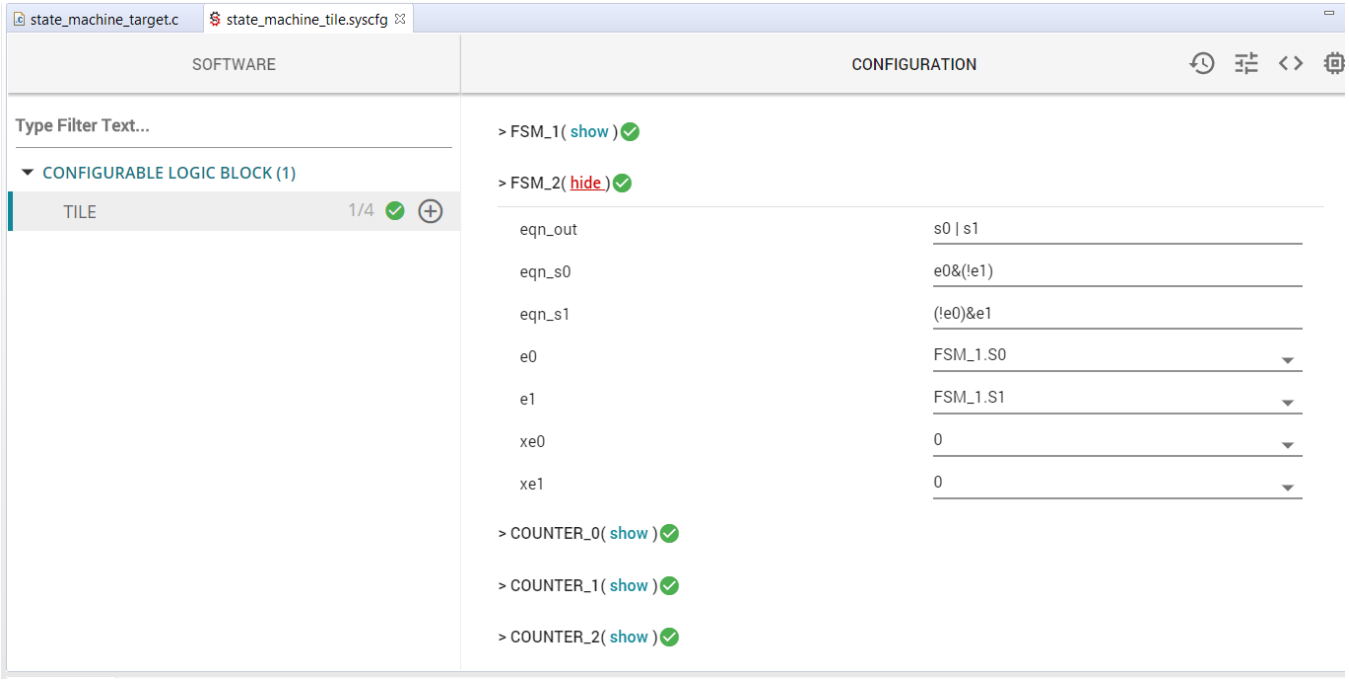


图 8. FSM2 配置

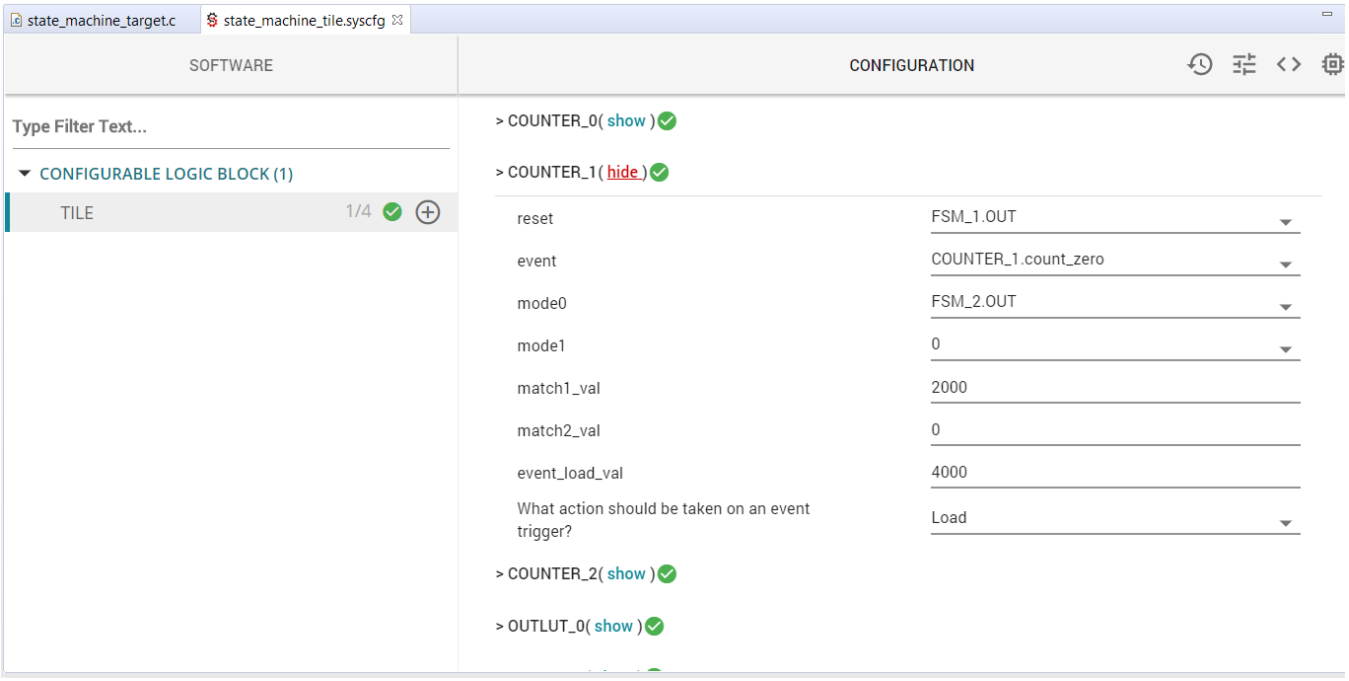


图 9. COUNTER1 配置

Copyright © 2019, Texas Instruments Incorporated

生成 PWM 信号的最后一步是利用 FSM 模块，根据 COUNTER1 和 FSM2 生成的事件与信号来产生 PWM 波形。该 PWM 信号由 FSM0 的 s0 生成。当 COUNTER1 发生 counter=ZERO 事件时，必须将 s0 清零为 LOW；当 COUNTER1 发生 counter=match1_val 事件时，必须将 s0 置位为 HIGH。

图 10 展示了 FSM0 的 SysConfig 配置。

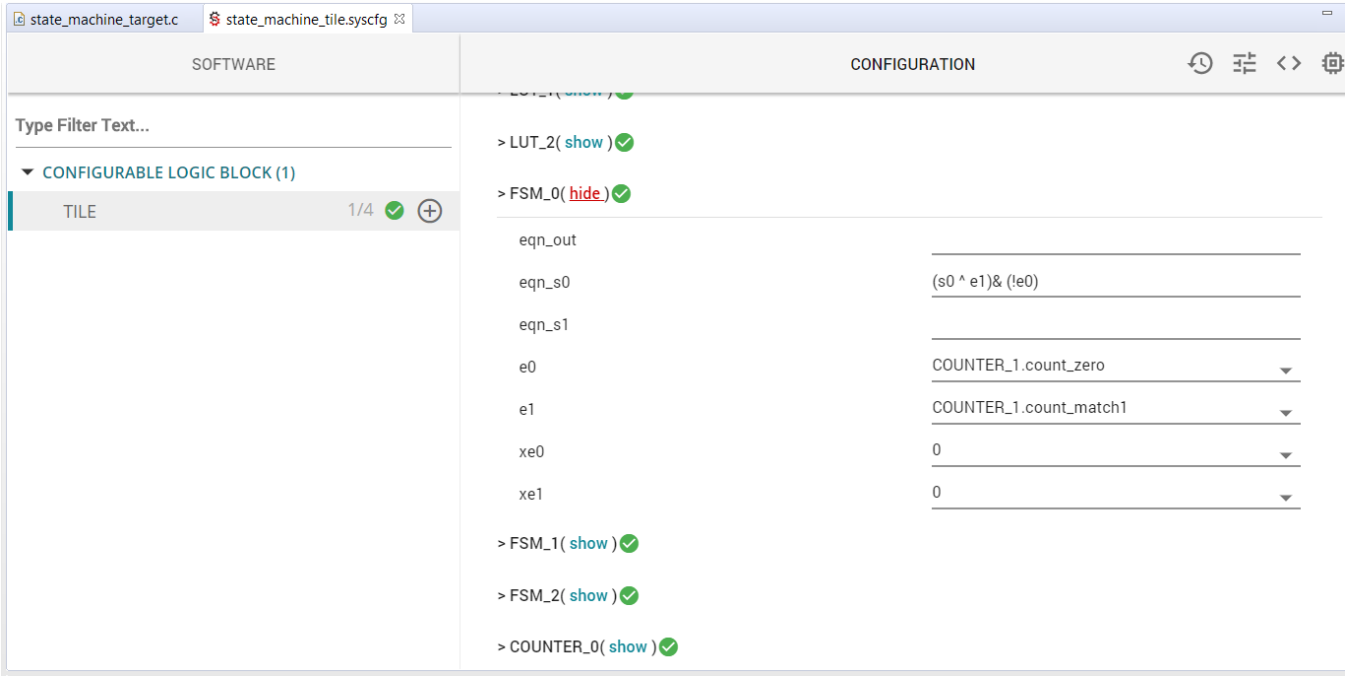


图 10. FSM0 配置

Copyright © 2019, Texas Instruments Incorporated

最后，将 PWM 信号分为两路，分别用于“正在关闭”和“正在打开”状态，并由 OUTLUT0 和 OUTLUT 2 输出。系统处于“正在打开”状态时，OUTLUT0 输出 PWM 信号，OUTLUT2 保持 LOW；系统处于“正在关闭”状态时，OUTLUT2 输出 PWM 信号，OUTLUT0 保持 LOW。图 11 展示了 OUTLUT0 和 OUTLUT2 的 SysConfig 配置。

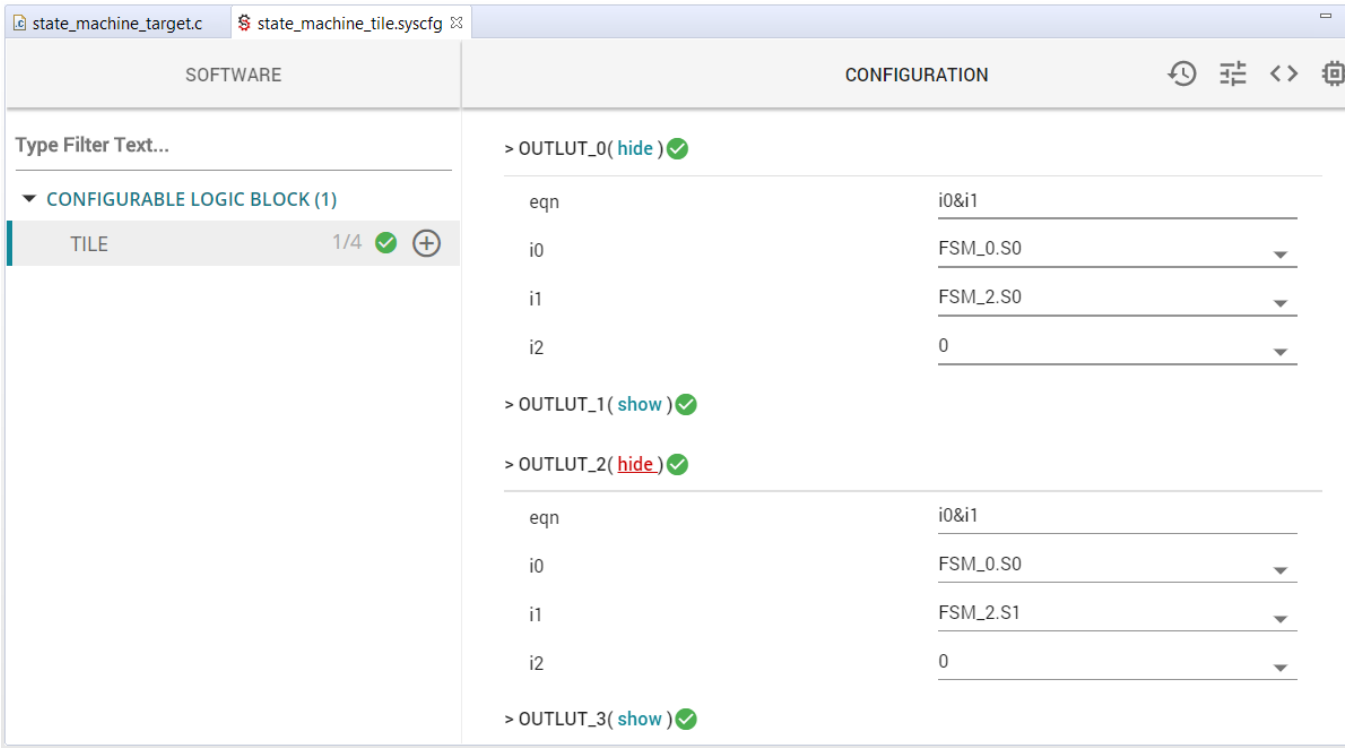


图 11. OUTLUT0 和 OUTLUT2 配置

下一步可随时使用 C28x 内核和 HLC 子模块更新 PWM 的周期和占空比。

7 修改 PWM 周期和占空比

要修改 CLB 生成的 PWM 信号的周期和占空比，需要更新 COUNTER1 的 load_val 和 match1_val。C28x 内核可访问 COUNTER1 中的这两个寄存器，并按需更新其值。为避免不可预期的行为，最好在 COUNTER1 的 counter=ZERO 事件处更新 load_val 和 match1_val。本设计会在 C28x 内核发出更新请求后的下一个 counter=ZERO 事件处更新计数器值。数据通过 HLC 子模块和 PUSH-PULL 接口装载：该接口是一个 4 深度 FIFO，供 C28x 内核与 HLC 子模块传输数据。C28x 内核将新的周期和占空比值写入 PUSH-PULL FIFO，HLC 在下一个 counter=ZERO 事件处读取这些值并更新 COUNTER1 寄存器。C28x 还必须向 HLC 指示 PUSH-PULL FIFO 中已有新值。该信号使用 GPREG 寄存器的 BIT2（0b00000100）生成；GPREG 的各个位连接到 CLB 逻辑单元（tile）的 BOUNDARY 输入。输入事件触发后，HLC 执行相应操作。

本设计使用 EVENT0 通知 HLC 读取 PUSH- PULL FIFO 并更新 PWM 值。检测到 HLC 的 EVENT0 上升沿时，HLC 会执行指定指令。由于 C28x 内核必须通知 HLC 在下一个 COUNTER1 counter=ZERO 事件处开始更新 PWM 值，因此 HLC 的 EVENT0 输入必须是 COUNTER1 counter=ZERO 与 BOUNDARY IN2 进行 AND 运算后的信号。LUT2 对这两个信号执行 AND 运算，再将其输出连接到 HLC 的 EVENT0 输入。

Copyright © 2019, Texas Instruments Incorporated

图 12 展示了 LUT2 的 SysConfig 配置。

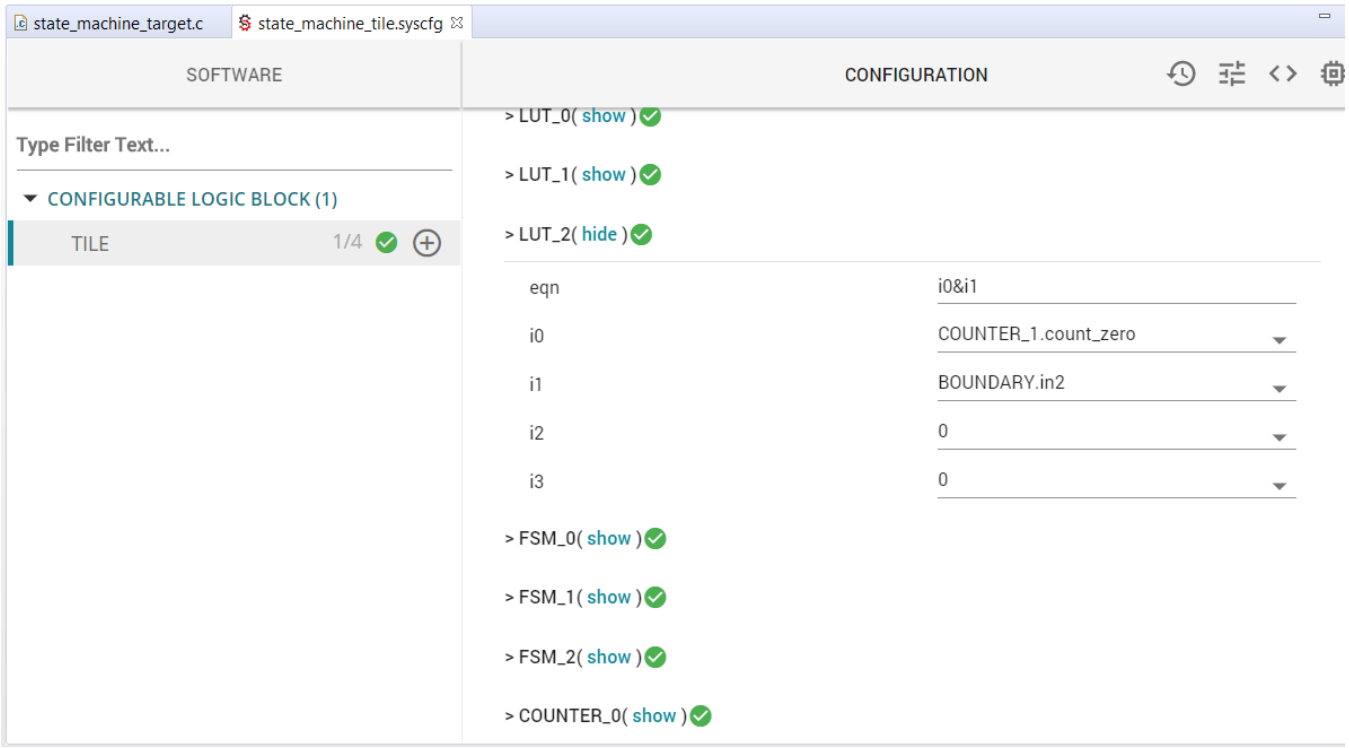


图 12. LUT2 配置

接下来定义 HLC 指令。C28x 内核先将新的周期值、再将新的占空比值写入 FIFO。HLC 必须按相同顺序从 FIFO 中 PULL 这些值。首先，HLC 通过 PULL 指令将周期值读入 COUNTER1 的活动计数器寄存器（该值也会写入 load_val，详见下一步）。随后从 FIFO PULL 的值必须写入 COUNTER1 的 match1_val。不过，PULL 指令无法将值写入计数器子模块的 match1_val。只有 MOV_T1 和 MOV_T2 指令可以写入计数器的匹配值；这两条指令分别访问 match1_val 和 match2_val。HLC 使用 PULL 指令将 FIFO 中的值读入 R1 寄存器，随后使用 MOV_T1 指令将 R1 的值写入 COUNTER1 的 match1_val。

HLC 执行的最后一条指令是 INTR。INTR 指令会在 C28x 内核上生成中断，通知该内核更新已完成。此时 FIFO 已空，可以再次使用。C28x 内核还会清除 GPREG BIT2，确保下一个 counter=ZERO 事件不会再次触发 HLC 事件（否则 FIFO 为空时仍会读取，导致下溢）。INTR 指令需要一个名为“interrupt tag”的参数。C28x 内核端会读取此标记，以确定中断原因。HLC 可在 tile 中的不同事件处生成多个中断，但本设计仅使用一个中断。INTR 指令使用的 interrupt tag 为“1”；C28x 内核在处理中断服务程序时会确认该标记为“1”。

Copyright © 2019, Texas Instruments Incorporated

图 13 展示了 HLC 子模块的 SysConfig 配置。

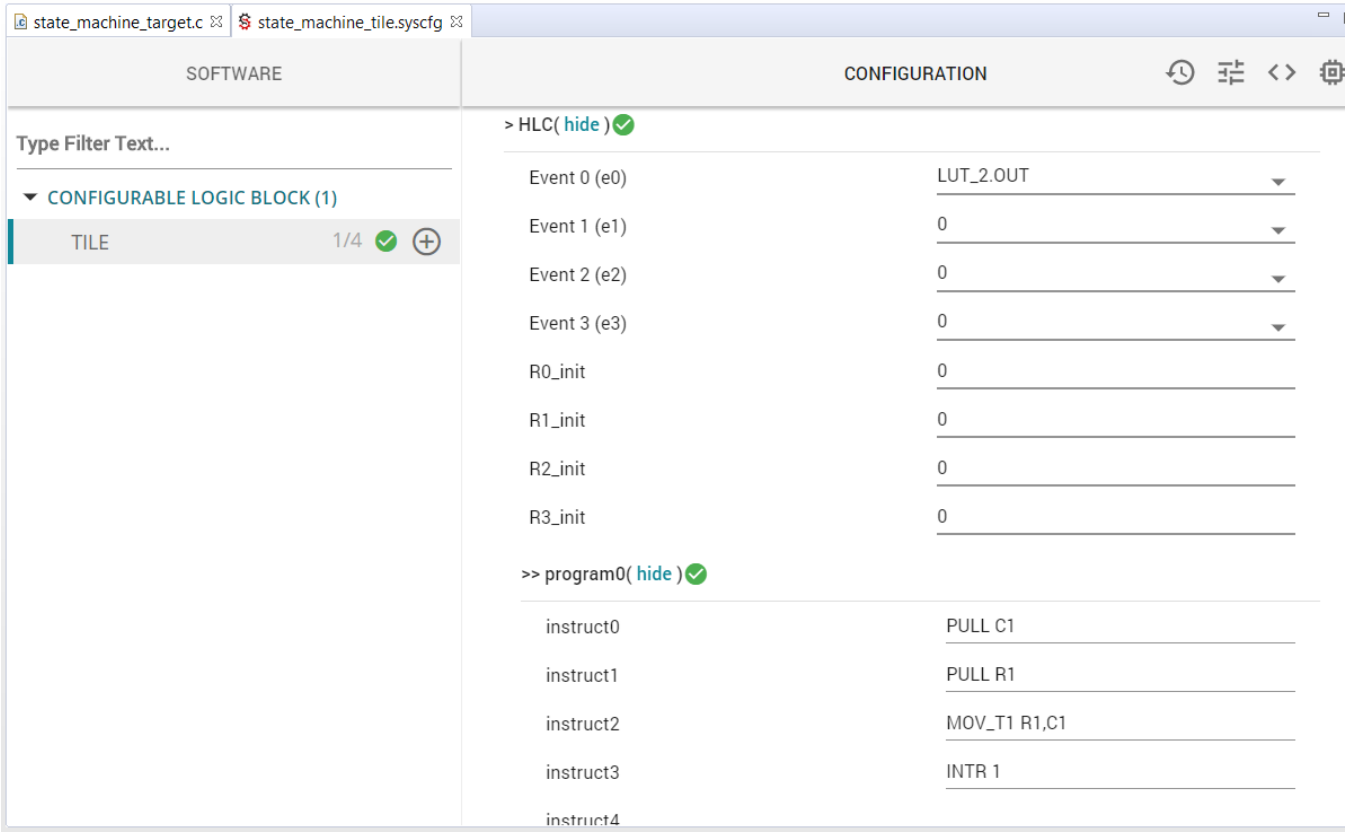


图 13. HLC 配置

本部分设计尚需完成的最后一项，是更新 COUNTER1 的 load_val。HLC 无法访问该寄存器，因此 C28x 必须在设置 GPREG BIT2 之前更新此值；否则 PWM 周期不会更新。以下代码展示 C28x 内核如何更新 PWM 周期和占空比，并在收到中断后清除 GPREG。

```
void updateClbPwm(uint32_t period, uint32_t duty) {if (canUpdate) { canUpdate = 0; CLB_writeInterfa
ce(CLB1_BASE, CLB_ADDR_COUNTER_1_LOAD, period); HWREG(CLB1_BASE + CLB_LOGIC
CTL + CLB_O_BUF_PTR) = 0U; HWREG(CLB1_BASE + CLB_DATAEXCH + CLB_O_PULL(0))
= period; HWREG(CLB1_BASE + CLB_DATAEXCH + CLB_O_PULL(1)) = duty; CLB_setGPREG(
CLB1_BASE, 1 << TRIGGER_PWM_UPDATE_SHIFT); } }__interrupt void clb1ISR(void) { uint16_t
tag = CLB_getInterruptTag(CLB1_BASE); if (tag == UPDATE_PWM_COMPLETED_TAG) { canUp
date = 1; CLB_setGPREG(CLB1_BASE, 0); }CLB_clearInterruptTag(CLB1_BASE); Interrupt_clearA
CKGroup(INTERRUPT_ACK_GROUP5); }
```


Copyright © 2019, Texas Instruments Incorporated

第 9 节介绍如何使用 XBAR 模块将信号输入和输出 CLB 逻辑单元（tile）。

8 完整设计

图 14 展示了完整的设计框图。子模块之间的连接有助于全面了解 CLB 逻辑单元（tile）要实现的功能。

Copyright © 2019, Texas Instruments Incorporated

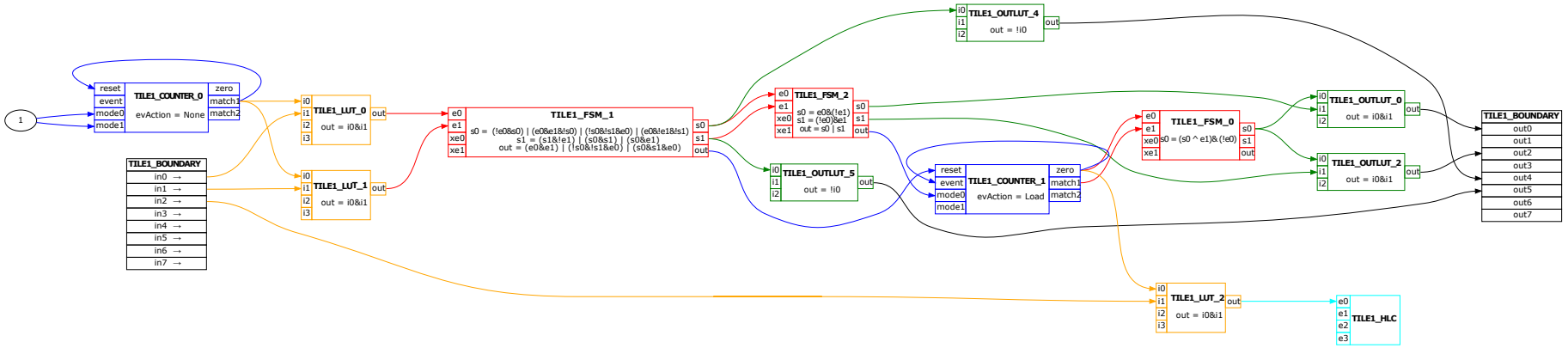


图 14. 完整设计框图

Copyright © 2019, Texas Instruments Incorporated

选择子模块时务必谨慎。例如，由于硬件限制，不能在上述设计中用 FSM0 替代 FSM1，因为 LUT1 的输出无法被选作 FSM0 的 e1 输入。这属于无效配置。*TMS320F2837xD Dual-core Delfino™ Microcontrollers Technical Reference Manual* 列出了所有此类无效设置。硬件会断开某些输入组合，以避免逻辑环路。图 15 展示了 CLB SysConfig 工具针对这一设计错误生成的警告。

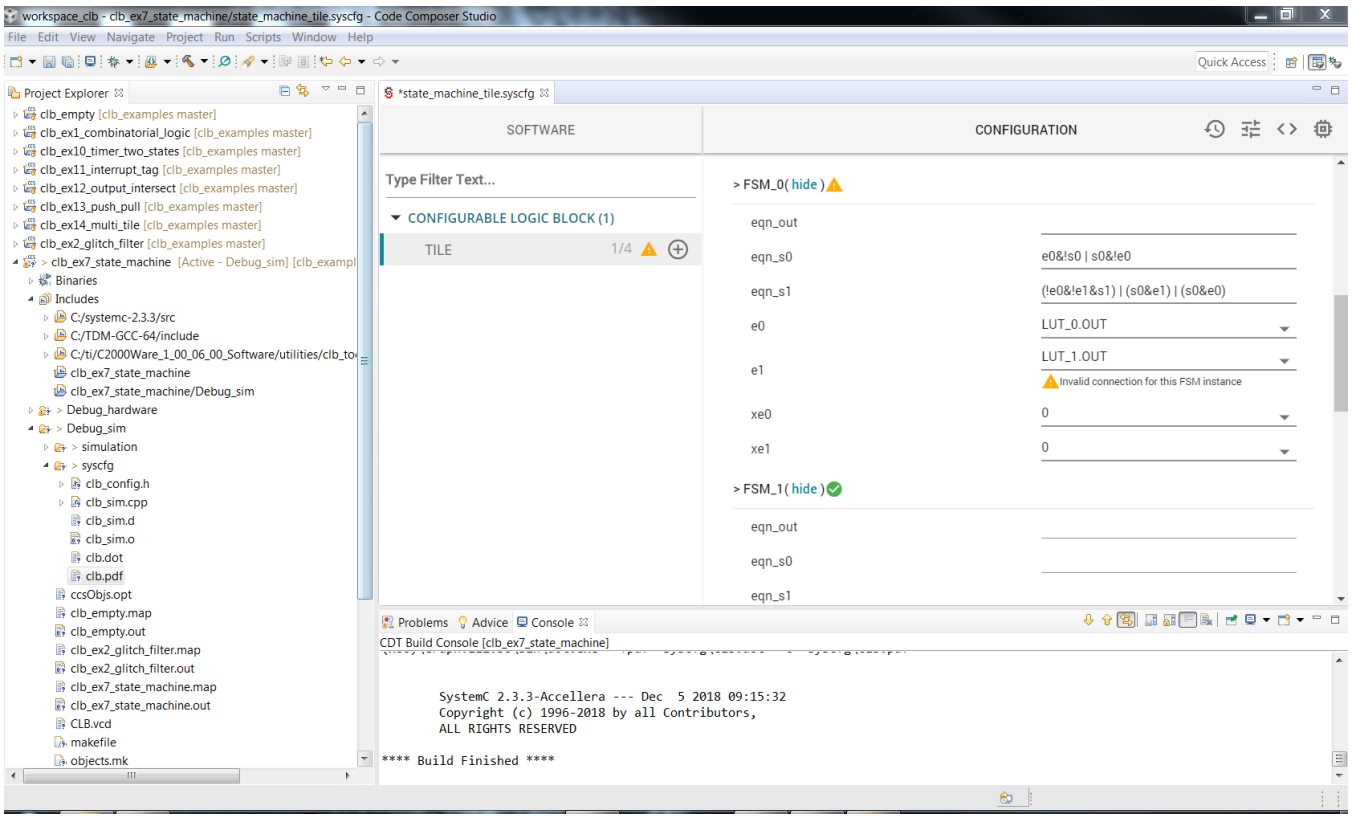


图 15. 此实例中的无效连接

9 输入 X-BAR、输出 X-BAR 和 CLB X-BAR

X-BAR 模块可将外部信号输入 CLB，也可将信号从 CLB 输出。本节介绍如何为本应用报告中的 CLB 逻辑单元（tile）配置 X-BAR 模块（INPUT X-BAR、OUTPUT X-BAR 和 CLB X-BAR）。

9.1 使用 X-BAR 将信号导入 CLB Tile

要将 GPIO 信号输入 CLB 的 BOUNDARY INz（其中 z 为 0–7 之间的任意数值），必须配置 INPUT X-BAR 和 CLB X-BAR。将 GPIO 信号输入 CLB 的步骤如下：

1. 按常规方法配置 GPIO：
   a. 设置方向：INPUT/OUTPUT。b. 启用或禁用 PULL-UP。c. 设置其他 GPIO 配置。
2. 使用 INPUT X-BAR（例如 INPUTx，其中 x 为 1 到可用 INPUT 数量上限之间的任意值）并选择所需的 GPIO。
3. 在 CLB 模块中，为 BOUNDARY INz 选择 GLOBAL 输入，而不是 LOCAL 输入。4. 使用 CLB X-BAR，将 INPUT X-BAR 中的 INPUTx 选作 AUXSIGy（其中 y 为 0–7 之间的任意值）。

Copyright © 2019, Texas Instruments Incorporated

5. 将 AUXSIGy 选作 BOUNDARY INz 的 GLOBAL 输入。
6. 禁用 GPREG 输入并启用外部输入（GLOBAL input = AUXSIGy input = INPUTx input = GPIO input）。

图 16 展示了如何从 GLOBAL 输入、LOCAL 输入或 GPREG 位中选择 CLB 的 BOUNDARY 输入。

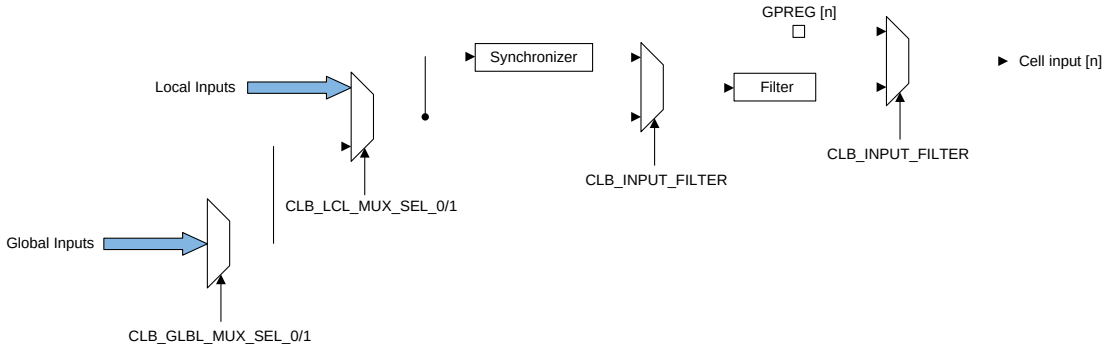


图 16. CLB BOUNDARY 输入多路选择

以下代码示例展示如何使用器件 DriverLib 驱动库完成前述配置。

```
//// Configure GPIO4 for Button //GPIO_setPinConfig(GPIO_4_GPIO4); GPIO_setDirectio
nMode(4, GPIO_DIR_MODE_IN); GPIO_setPadConfig(4, GPIO_PIN_TYPE_PULLUP); //
// Configure Input-XBAR INPUT1 to GPIO4 //XBAR_setInputPin(XBAR_INPUT1, 4); ////
Configure CLB-XBAR AUXSIG0 as INPUT1 //XBAR_setCLBMuxConfig(XBAR_AUXSI
G0, XBAR_CLB_MUX01_INPUTXBAR1); XBAR_enableCLBMux(XBAR_AUXSIG0, X
BAR_MUX01);
```

```
CLB_configLocalInputMux(CLB1_BASE, CLB_IN0, CLB_LOCAL_IN_MUX_GLOBAL_IN); CLB_configGl
obalInputMux(CLB1_BASE, CLB_IN0, CLB_GLOBAL_IN_MUX_CLB_AUXSIG0); CLB_configGPInputMu
x(CLB1_BASE, CLB_IN0, CLB_GP_IN_MUX_EXTERNAL);
```

9.2 使用 X-BAR 从 CLB Tile 导出信号

要将 CLB 的信号（BOUNDARY OUTx）输出到 GPIO，可对 OUT4 和 OUT5 使用 OUTPUT X-BAR；OUT0–3 和 OUT6–7 则可在 GPIO 处拦截特定外设输出。

若使用 OUT4 和 OUT5，请按以下步骤配置 OUTPUT X-BAR：

1. 按常规方法配置 GPIO。
   a. 设置方向：INPUT/OUTPUT。b. 启用/禁用 PULL-UP。c. 设置其他 GPIO 配置。

Copyright © 2019, Texas Instruments Incorporated

2. 配置 PINMUX 以使用 OUTPUT X-BAR。

以下代码示例展示如何使用器件 DriverLib 驱动库完成前述配置。

```
//// Configure GPIO31 for OUTPUTXBAR8 //GPIO_setPinConfig(GPIO_31_OUTPUTXBAR
8); GPIO_setDirectionMode(31, GPIO_DIR_MODE_OUT); GPIO_setPadConfig(31, GPIO_PI
N_TYPE_STD); //// Configure OUTPUT-XBAR OUTPUT8 as CLB1_OUT4 //XBAR_setOutp
utMuxConfig(XBAR_OUTPUT8, XBAR_OUT_MUX01_CLB1_OUT4); XBAR_enableOutpu
tMux(XBAR_OUTPUT8, XBAR_MUX01); //// Configure GPIO34 for OUTPUTXBAR1 //GPI
O_setPinConfig(GPIO_34_OUTPUTXBAR1); GPIO_setDirectionMode(34, GPIO_DIR_MOD
E_OUT); GPIO_setPadConfig(34, GPIO_PIN_TYPE_STD); //// Configure OUTPUT-XBAR O
UTPUT1 as CLB1_OUT5 //XBAR_setOutputMuxConfig(XBAR_OUTPUT1, XBAR_OUT_M
UX03_CLB1_OUT5); XBAR_enableOutputMux(XBAR_OUTPUT1, XBAR_MUX03);
```

若使用外设输出拦截功能（CLB 逻辑单元（tile）的 OUT0–3 和 OUT6–7）：
1. 按常规方法为相应外设配置 PINMUX。a. 示例：为 EPWM1A 配置 GPIO0。2. 使用 OUT_EN 寄存器使能 CLB 的 OUT0–3 和 OUT6–7 输出。

此时 GPIO 输出为 CLB OUT，而不是外设输出。TRM 中的 Peripheral Signal Multiplexer 表列出了各外设输出对应的 CLB OUT。以下代码示例展示如何使用器件 DriverLib 驱动库完成前述配置。

```
//// Configure GPIO0 for EPWM1A which will be overriden by CLB0_OUT0 //GPIO_setPinConfi
g(GPIO_0_EPWM1A); GPIO_setDirectionMode(0, GPIO_DIR_MODE_OUT); GPIO_setPadCon
fig(0, GPIO_PIN_TYPE_STD); //// Configure GPIO1 for EPWM1B which will be overriden by C
LB0_OUT1 //GPIO_setPinConfig(GPIO_1_EPWM1B); GPIO_setDirectionMode(1, GPIO_DIR_
MODE_OUT); GPIO_setPadConfig(1, GPIO_PIN_TYPE_STD);
```

最后，必须使能 CLB 逻辑单元（tile）的输出。以下代码示例使用器件 DriverLib 驱动库完成此操作。

```
CLB_setOutputEnableMask(CLB1_BASE, 1 << 0 | 1 << 2)                ;
```


Copyright © 2019, Texas Instruments Incorporated

10 运行示例项目

本节介绍如何测试本文所设计系统对应的 Code Composer Studio™ 示例项目。该示例已在 TMS320F28379D LaunchPad 上进行测试。

10.1 设置与连接

图 17 展示了 BOUNDARY IN0 和 IN1 所用的引脚、分别对应关闭和打开状态的 PWM 输出，以及用于显示 FSM1 子模块当前状态的 LED。

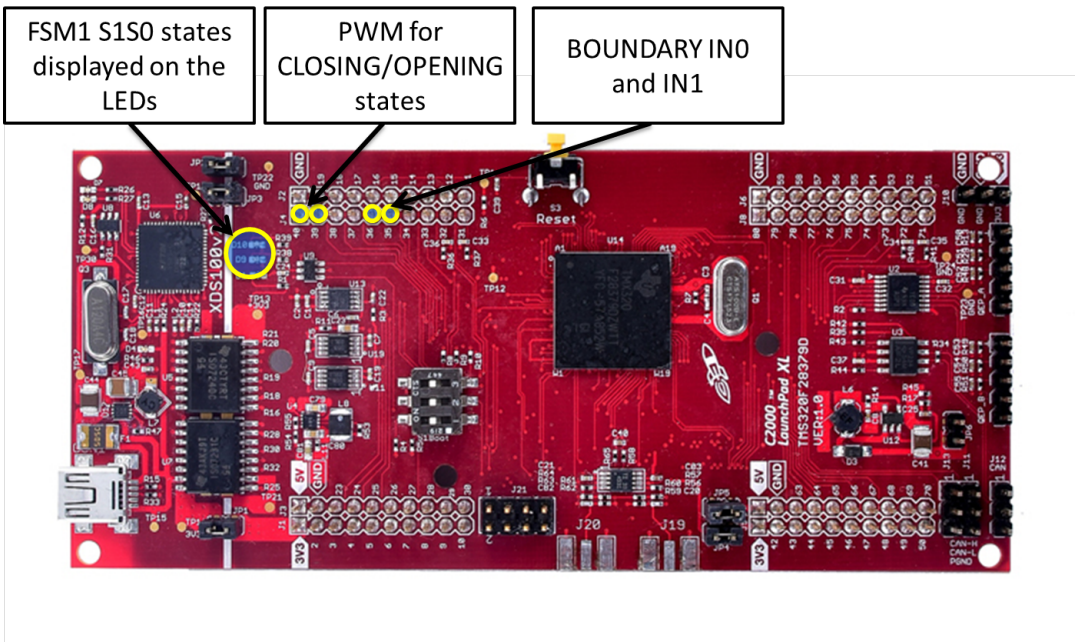


图 17. TSM320F28379D LaunchPad 连接示意图

10.2 测试状态及其转换

运行代码前，先用跳线将 BOUNDARY IN0 和 IN1 都接至 GND。这些 GPIO 已启用内部上拉。断开 GND 连接后，引脚电平将变为 HIGH。将示波器探头分别接到 GPIO0 和 GPIO1，以观察系统处于关闭或打开状态时的 PWM 信号。以下各节将说明如何逐步切换状态并在 LaunchPad 上查看结果。

10.2.1 步骤 1

按以下步骤查看系统从打开状态切换到正在关闭状态的过程：

1. 运行内核。两个 LED 均熄灭，示波器上也没有 PWM 信号。2. 断开 BOUNDARY IN0 与 GND 的连接。a. 状态机从打开状态切换到正在关闭状态。对应关闭状态的 GPIO 上会出现 PWM 信号。状态 LED 显示为 OFF-ON，对应 S1 = 0、S0 = 1。3. 再次将 BOUNDARY IN0 接至 GND。此操作不会产生任何变化。

10.2.2 步骤 2

按以下步骤查看系统从正在关闭状态切换到关闭状态的过程：

1. 断开 BOUNDARY IN1 与 GND 的连接。
   a. 状态机从正在关闭状态切换到关闭状态，PWM 信号停止。状态 LED 显示为 ON-ON，对应 S1 = 1、S0 = 1。
2. 再次将 BOUNDARY IN1 接至 GND。此操作不会产生任何变化。

Copyright © 2019, Texas Instruments Incorporated

10.2.3 步骤 3

按以下步骤操作 ，查看系统从关闭状态切换到正在打开状态的过程 ：

1. 断开 BOUNDARY IN0 与 GND 的连接。a. 状态机从关闭状态切换到正在打开状态。对应打开状态的 GPIO 上会出现 PWM 信号。状态 LED 显示为 ON-OFF，对应 S1 = 1、S0 = 0。2. 再次将 BOUNDARY IN0 接至 GND。此操作不会产生任何变化。

10.2.4 步骤 4

按以下步骤查看系统从正在打开状态切换到打开状态的过程:

1. 断开 BOUNDARY IN1 与 GND 的连接。a. 状态机从正在打开状态切换到打开状态，PWM 信号停止。状态 LED 显示为 OFF-OFF，对应 S1 = 0、S0 = 0。2. 再次将 BOUNDARY IN1 接至 GND。此操作不会产生任何变化。

10.2.5 步骤 5

在最后一项测试中，断开 IN0 和 IN1 与 GND 的连接，使系统状态在正在关闭和正在打开之间持续切换。状态切换会使状态值和 PWM 信号以采样计数器的频率不断翻转。

10. 3 测试 PWM 周期和占空比

最后要测试的是运行时修改 PWM 周期和占空比的功能。请按以下步骤测试:

1. 运行代码并打开 Expressions 窗口。2. 添加“clbPwmUpdateNow”、“clbPwmDuty”和“clbPwmPeriod”。3. 启用 Auto Refresh。

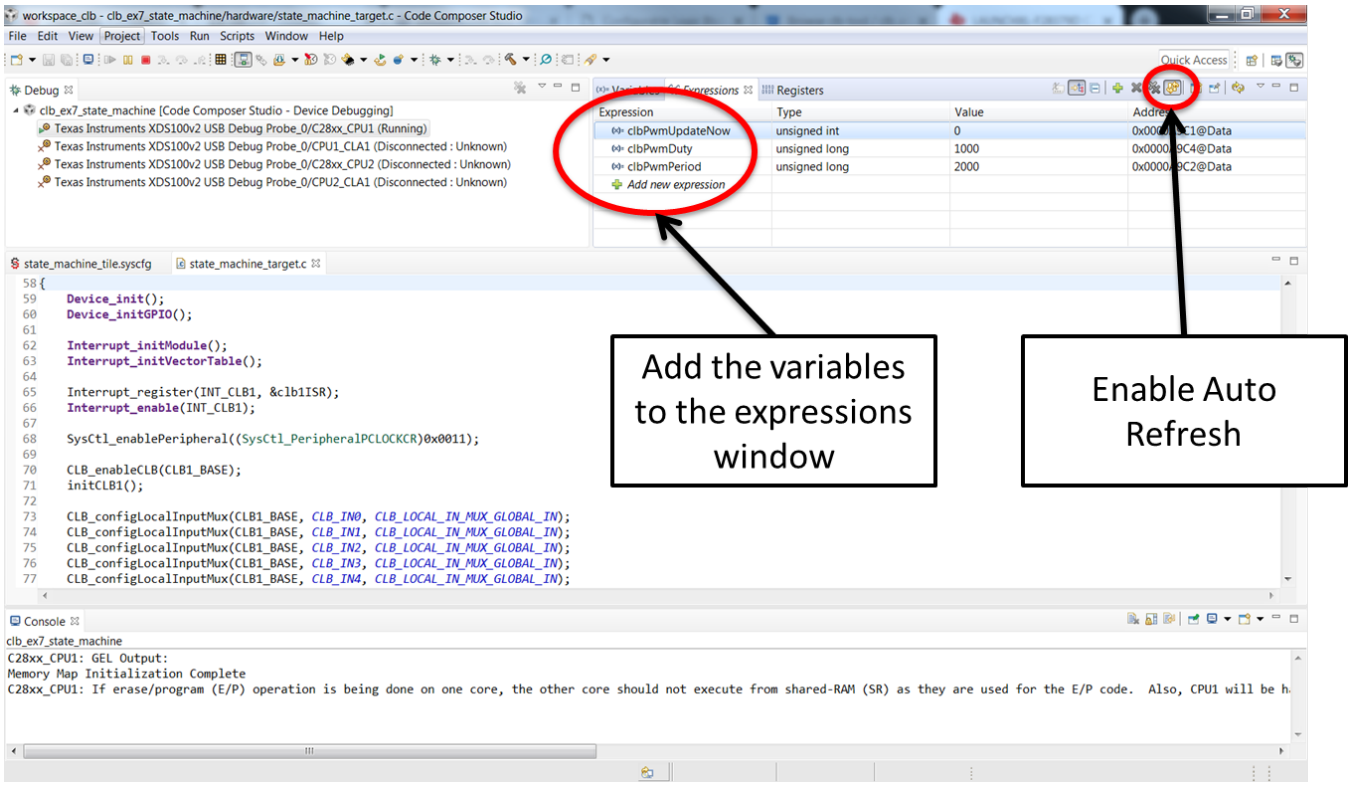


图 18. 在 Expressions 窗口中修改 PWM 周期和占空比

Copyright © 2019, Texas Instruments Incorporated

4. 单击 clbPwmDuty 和 clbPwmPeriod 的值并将其更新为新值。5. 单击 clbPwmUpdateNow 的值，并输入“1”。6. PWM 参数随即更新，可在示波器上观察到变化。

图 19 和图 20 展示了将 PWM 占空比设为 1000、周期设为 2000 的示例。

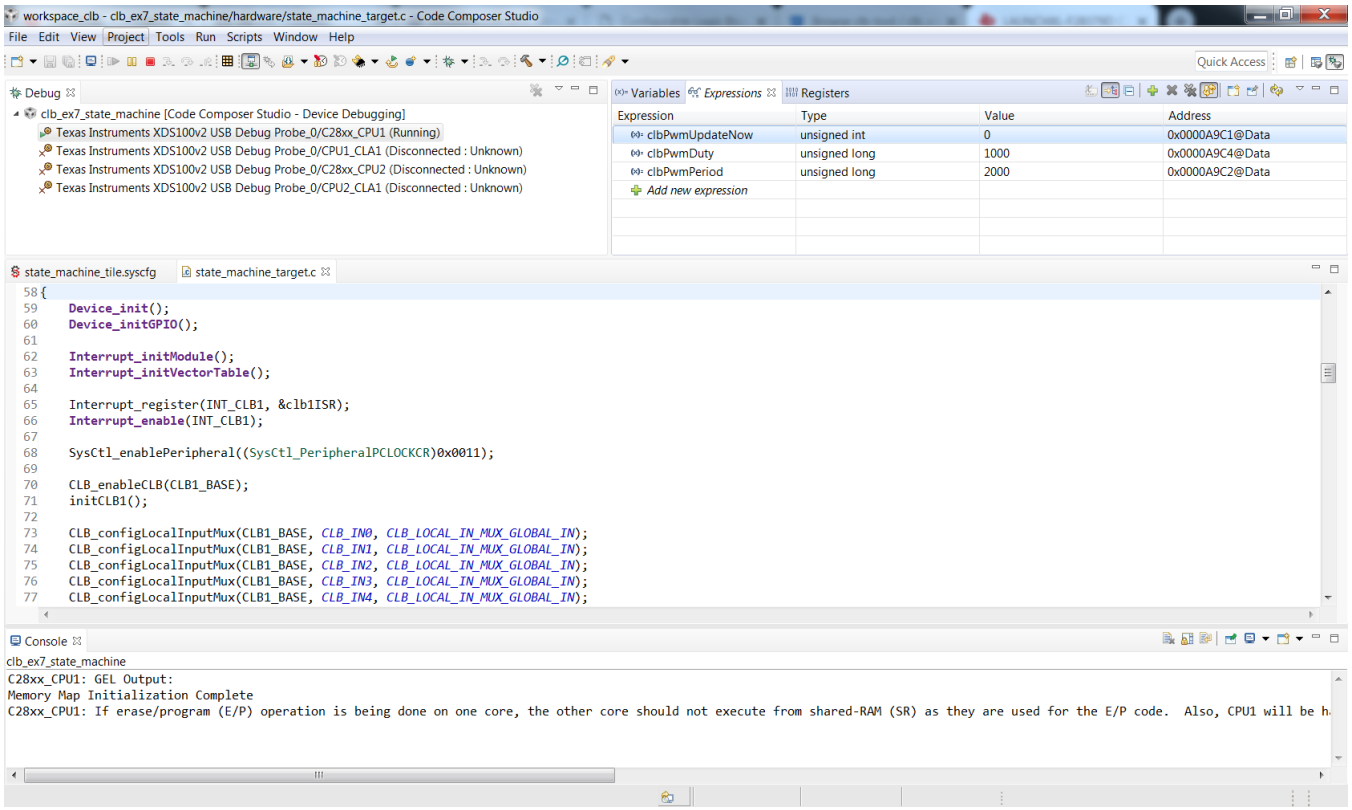


图 19. PWM 占空比 1000，周期 2000

Copyright © 2019, Texas Instruments Incorporated

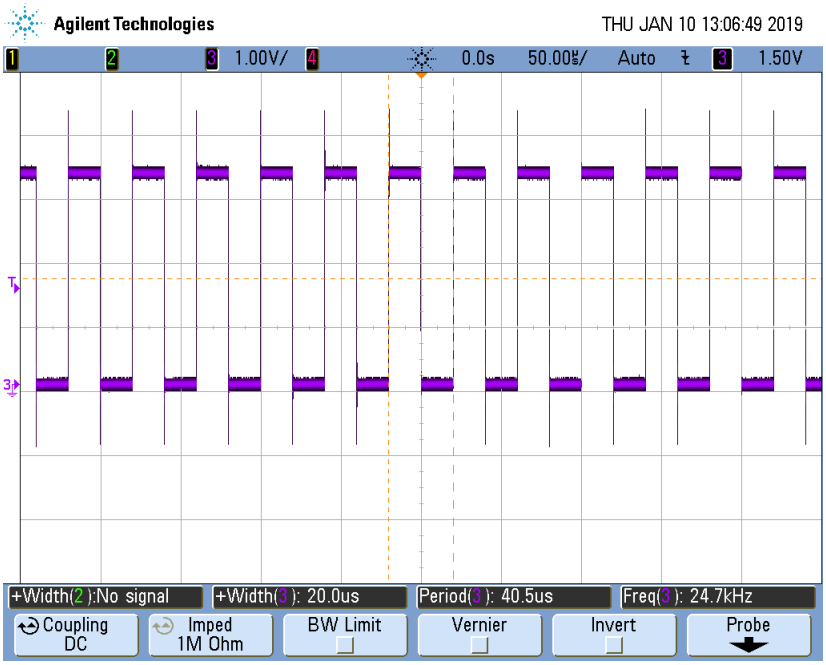


图 20. 示波器波形——PWM 占空比 1000，周期 2000

图 21 和图 22 展示了将 PWM 占空比设为 3000、周期设为 4000 的示例。

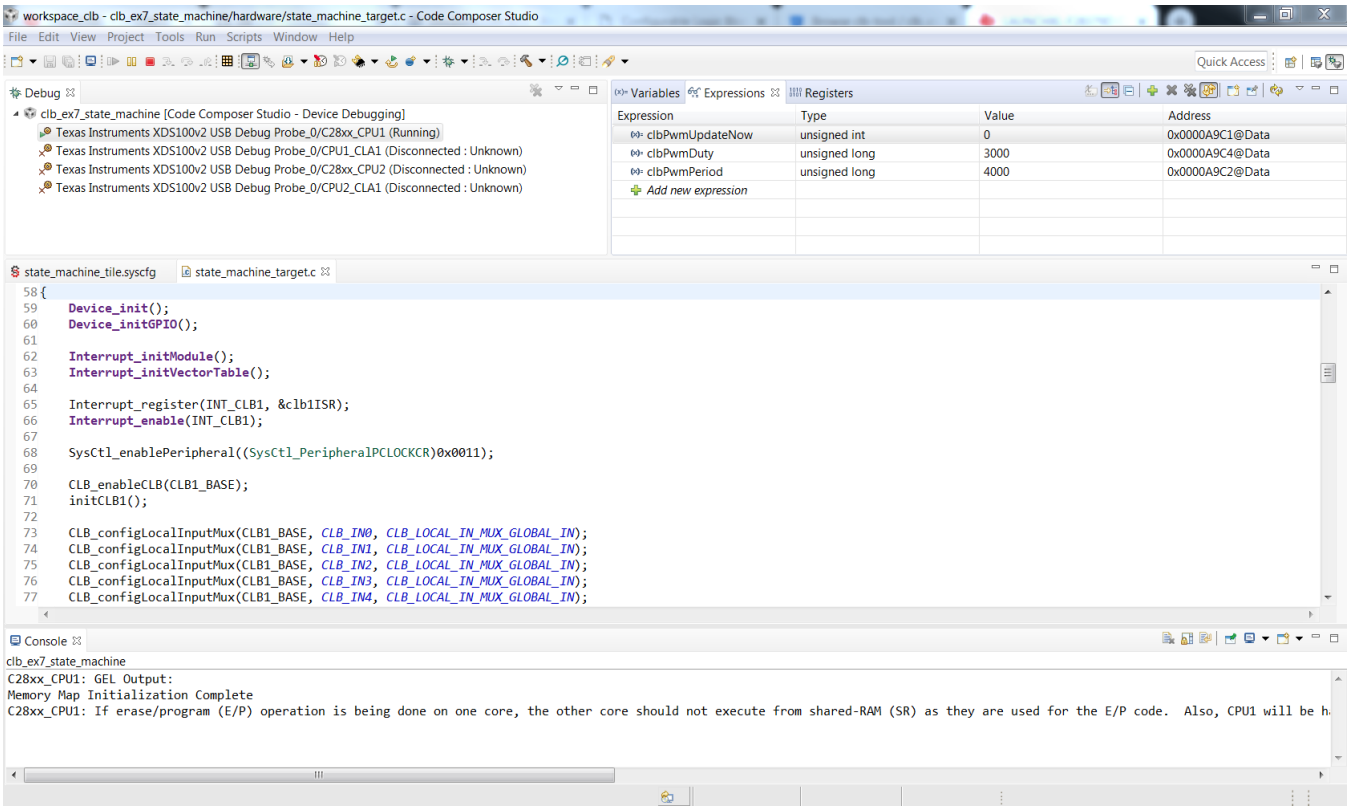


图 21. PWM 占空比 3000，周期 4000

Copyright © 2019, Texas Instruments Incorporated

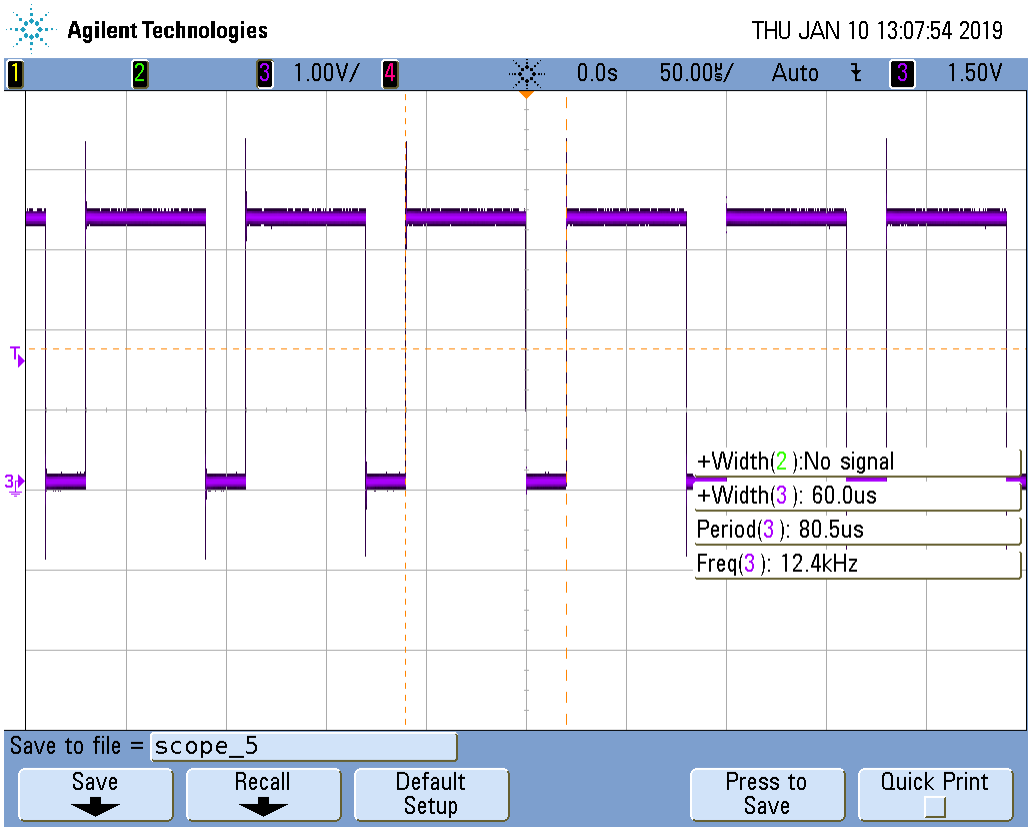


图 22. 示波器波形——PWM 占空比 3000，周期 4000

11 总结

CLB 模块是一种功能强大且高度灵活的外设，可用于构建高性能设计，使其独立运行或与 C28x 内核协同工作。本文还展示了 CLB SysConfig Tool 的用法，以及 XBAR 模块导入信号到 CLB 模块或从 CLB 模块导出信号所需的配置。

12 参考资料

- *TMS320F2837xD Dual-Core Delfino™ Microcontrollers Technical Reference Manual*
- *TMS320F28004x Piccolo Microcontrollers Technical Reference Manual*

Copyright © 2019, Texas Instruments Incorporated

重要声明与免责声明

TI 按“原样”提供技术和可靠性数据（包括数据手册）、设计资源（包括参考设计）、应用或其他设计建议、Web 工具、安全信息及其他资源，并且不保证其不存在任何缺陷。TI 明确声明不承担任何明示或默示保证，包括但不限于适销性、对特定用途的适用性以及不侵犯第三方知识产权的默示保证。

这些资源面向使用 TI 产品进行设计的专业开发人员。您应自行负责：(1) 为应用选择适当的 TI 产品；(2) 设计、验证并测试应用；以及 (3) 确保应用符合适用标准以及其他安全、安保、监管或其他要求。

这些资源如有更改，恕不另行通知。TI 仅允许您将这些资源用于开发采用本资源所述 TI 产品的应用。禁止以其他方式复制或展示这些资源。TI 不授予任何其他 TI 知识产权或任何第三方知识产权许可。对于因您使用这些资源而产生的任何索赔、损害、成本、损失和责任，TI 概不负责；您应对 TI 及其代表承担全额赔偿责任。

TI 产品受 TI 销售条款或其他适用条款约束；相关条款可在 ti.com 上查阅，也可能随 TI 产品一并提供。TI 提供这些资源不会扩大或以其他方式改变 TI 产品适用的保证或保证免责声明。

TI 反对并拒绝接受您可能提出的任何附加条款或不同条款。 重要声明

Mailing Address: Texas Instruments, Post Office Box 655303, Dallas, Texas 75265
Copyright © 2022, Texas Instruments Incorporated